

Docker Compose

Multi-Container Apps · Services · Networks · Depends-On

■ Analogy

- Docker Compose is like a restaurant's opening checklist. Instead of manually starting the kitchen (database), bar (cache), and dining room (app) in the right order, you hand the checklist to a manager (Compose) who starts everything in the correct order, connects the rooms, and manages the whole operation.

Key Terms

docker-compose.yml	YAML file defining all services, networks, and volumes for an application
Service	A containerized component — e.g., 'app', 'db', 'redis' — each maps to one image
depends_on	Controls startup order — but does NOT wait for readiness. Use healthchecks for that
healthcheck	Defines a command to check if service is ready. depends_on condition: service_healthy waits for it
environment	Sets env vars in the container. Can inline values or reference .env file
profiles	Groups services — docker compose --profile debug up to start optional services
extends	Inherits config from another service or file — reduces duplication
compose watch	Auto-rebuilds/syncs containers when source files change — replaces bind mounts for dev
--project-name	Namespaces all resources — useful when running multiple compose files on same host
Override file	docker-compose.override.yml — automatically merged, used for local dev customizations

Full docker-compose.yml Example

ShopVerse docker-compose.yml

```
version: '3.9'
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: shopverse
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5
  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/shopverse
      SPRING_REDIS_HOST: redis
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
volumes:
  pgdata:
```

Useful Compose Commands

Command	What It Does
<code>docker compose up -d</code>	Start all services in background
<code>docker compose down -v</code>	Stop and remove containers + volumes
<code>docker compose logs -f app</code>	Follow logs for 'app' service
<code>docker compose ps</code>	List service status
<code>docker compose exec app bash</code>	Shell into running 'app' container
<code>docker compose build --no-cache</code>	Rebuild images ignoring cache
<code>docker compose scale app=3</code>	Run 3 instances of 'app' service

■ Real-World: Shopify

- Shopify uses Docker Compose for local development across all engineering teams. A single docker compose up starts the Rails app, MySQL, Redis, Kafka, and supporting services — giving every engineer an identical environment regardless of their OS, eliminating setup docs that were always out of date.